

Convex Hull Algorithms for Piecewise Linear-Quadratic Functions in Computational Convex Analysis

Bryan Gardiner · Yves Lucet

Received: 9 August 2010 / Accepted: 10 August 2010 /
Published online: 25 August 2010
© Springer Science+Business Media B.V. 2010

Abstract Computing the convex envelope is a core operation in nonsmooth analysis that bridges the convex with the nonconvex world. Although efficient algorithms to compute fundamental transforms of convex analysis have been proposed over the years, they are limited to convex functions until an efficient algorithm becomes available to compute the convex envelope of a piecewise linear-quadratic function (of one variable) efficiently. We present two such algorithms, one based on maximum and conjugate computation that is easy to implement but has quadratic time complexity, and another based on direct computation that requires more work to implement but has optimal (linear time) complexity. We prove their time (and space) complexity, and compare their performances.

Keywords Computer-aided convex analysis · Computational convex analysis · Piecewise linear-quadratic functions · Quadratic spline · Moreau envelope · Legendre–Fenchel conjugate

Mathematics Subject Classifications (2010) Primary 90C25; Secondary 26A51 · 26B25 · 47H05 · 52A41

1 Introduction

Computational convex analysis, also named computer-aided convex analysis, focuses on designing efficient algorithms to compute fundamental transforms arising in convex analysis. The core idea is not to evaluate these operators at one specific value

B. Gardiner · Y. Lucet (✉)
Computer Science, I. K. Barber School #4, University of British Columbia Okanagan,
Kelowna, British Columbia V1V 1V7, Canada
e-mail: yves.lucet@ubc.ca

B. Gardiner
e-mail: khumba@interchange.ubc.ca

but to compute their full graphs. In that context, the early idea of efficient numerical computation of convex transforms can be traced back to [23]. However, most of the development of efficient algorithms in computational convex analysis started with a note on the Fast Legendre Transform (FLT) [5], which was further developed in [6, 17] (and independently in [24, 26]). It was later improved to optimal complexity in [18]. More recent algorithms to compute numerically the conjugate have been based on parametrization [13], the computation of the Moreau envelope [20], or manipulation of graphs [10]. More complex operators can be built by combining addition, scalar multiplication and conjugacy (or Moreau envelope computation) such as the proximal average operator [1, 2] whose challenging computation motivated further research in computational convex analysis [14, 22].

We refer to [21] for a survey of numerous applications of computational convex analysis in a wide range of fields (image processing, partial differential equations, thermodynamics, robotics, etc.).

The present work focuses on piecewise linear-quadratic (PLQ) functions, that have been well-known in convex analysis (the set of PLQ functions is closed under most common convex operators, see [25]). Their use in computational convex analysis was introduced in [22] with several algorithms, referred to as PLQ algorithms, to compute the core convex analysis operators (Legendre–Fenchel conjugate, Moreau envelope, addition, scalar multiplication) and their combinations like the proximal average. The PLQ algorithms have recently been improved by a new family of algorithms named GPH (for graph) algorithms [10]. The PLQ and GPH algorithms allow the computation of convex operators provided the initial function is convex. Should that assumption not be true, one is restricted to using the Linear-time Legendre-Transform (LLT) algorithm [18], which is intrinsically based on a piecewise linear model. Hence, all the advantages of PLQ functions outlined in [22] are not available.

To motivate our focus on PLQ functions, consider the computation of the proximal average $f := \left[(1 - \lambda) \left(f_0 + \frac{1}{2} |\cdot|^2 \right)^* + \lambda \left(f_1 + \frac{1}{2} |\cdot|^2 \right)^* \right]^* - \frac{1}{2} |\cdot|^2$ of two linear functions $f_0(x) := -x$ and $f_1(x) := x$ (from [22]). The critical issue is that the conjugate of a piecewise linear functions with full domain has bounded domain, which is compounded when composing several transforms. Let us assume the approximation is exact on the grid, and only consider the convergence required due to unbounded domains. Then using piecewise linear functions results in the following approximation

$$\begin{cases} +\infty & \text{if } |x| > b, \\ (2\lambda - 1)x - 2\lambda(1 - \lambda) & \text{if } 2(1 - \lambda) - b \leq x \leq b - 2\lambda, \\ \frac{\lambda}{2(1 - \lambda)}x^2 + \frac{\lambda + \lambda b - 1}{1 - \lambda}x + \frac{\lambda b(4\lambda + b - 4)}{2(1 - \lambda)} & \text{if } -b \leq x \leq 2(1 - \lambda) - b, \\ \frac{1 - \lambda}{2\lambda}x^2 + \frac{\lambda - b + \lambda b}{\lambda}x + \frac{b(4\lambda^2 + b - \lambda b - 4\lambda)}{2\lambda} & \text{if } b - 2\lambda \leq x \leq b \end{cases}$$

where f_0 (resp. f_1) is approximated with $-x + I_{[-b, b]}(x)$ (resp. $x + I_{[-b, b]}(x)$), and $b > 1$. The function we are looking for is $(2\lambda - 1)x - 2\lambda(1 - \lambda)$ which we obtain by considering the limit when $b \rightarrow +\infty$. This is in sharp contrast with the PLQ

framework that returns directly the exact result. As explained in [22], composing several transforms requires much technical knowledge when working with piecewise linear functions while PLQ functions are much easier to compose. The core reason is not the resulting increased accuracy but the fact that first the conjugate of a piecewise linear function with bounded domain has full domain, and second that the Moreau envelope of a piecewise linear function returns a PLQ function.

We extend the PLQ and GPH algorithms to nonconvex functions by proposing two new numerical algorithms to compute the convex envelope of a PLQ function. Then the conjugate operator can be applied to any nonconvex PLQ function by first computing its convex envelope, and then applying any of the algorithms from computational convex analysis previously studied. (The idea of using the convex envelope as a preliminary step was first introduced in [18].) More specifically, the scalar multiplication, addition, pointwise maximum, and pointwise minimum operations are (trivially) extended to nonconvex functions. Computing the conjugate of a nonconvex function becomes possible by first computing its convex envelope then applying a (convex) conjugate algorithm, while computing the Moreau envelope can be reduced to a conjugate computation. Hence all core convex operators are extended to nonconvex PLQ (univariate) functions as soon as one can compute the convex envelope.

The convex hull is a well-known operator in convex analysis that bridges the convex with the nonconvex world (see [12, III.1.4, X.1.5, X.2.6]). (We use the term convex hull for a set of points and the term convex envelope for a function.) It can be computed numerically using computational geometry algorithms such as the Beneath-Beyond algorithm [8], differential equations techniques (see [21] for a list of such numerical methods), or more algebraic methods [16]. Our focus is on using the special structure of PLQ functions to compute the convex envelope efficiently using ideas from computational geometry.

The paper is organized as follows. Section 2 introduces our notations, Section 3 presents our first algorithm named `plq_coSplit`, Section 4 introduces our second algorithm named `plq_coDirect`, and Section 5 contains the numerical validation and comparison of both algorithms. Finally, Section 6 concludes the paper with suggestion for future work.

2 Preliminaries

We adopt standard notations in the field of convex analysis [12]. A function $f : \mathbb{R}^d \rightarrow \mathbb{R} \cup \{-\infty, +\infty\}$ is said to be proper if f is never equal to $-\infty$ and f is not identically $+\infty$ (i.e. there is at least one point where f is finite). The domain of f , noted $\text{dom } f$, is the set of points where f is finite. The conjugate of a proper function f will be denoted by

$$f^*(s) = \sup_x [\langle s, x \rangle - f(x)]$$

where $\langle s, x \rangle$ denotes the standard euclidean dot product ($\|\cdot\|$ will denote the euclidean norm). The Moreau envelope for $\lambda > 0$ is written as

$$M_\lambda f(x) = \inf_y \left[f(y) + \frac{\|x - y\|^2}{2\lambda} \right].$$

Several computational convex analysis algorithms, which manipulate piecewise linear functions, can already handle nonconvex functions. The LLT and Parabolic Envelope (PE [9, 20]) algorithms can be used to compute the conjugate and the Moreau envelope of nonconvex piecewise linear functions with bounded domain. In fact, considering that the conjugate is always a convex function that depends only on the convex envelope

$$\text{co } f(x) = \inf \left\{ \sum_{i=1}^{d+1} \lambda_i f(y_i) : \lambda_i \geq 0, \sum_i \lambda_i = 1, x = \sum_{i=1}^{d+1} \lambda_i y_i \right\}, \quad (1)$$

algorithms restricted to convex functions can be extended to nonconvex functions by first convexifying the function i.e. computing its convex envelope, then computing its conjugate since $(\text{co } f)^* = f^*$ (this is the principle of the LLT algorithm). That approach is the reverse of the usual method of computing twice the conjugate to obtain the convex envelope since $\text{co } f = (f^*)^*$.

The Moreau envelope (resp. conjugate) can then be easily deduced from the conjugate (resp. Moreau envelope) by using the following formulas (see [19])

$$M_\lambda f(x) = \frac{\|x\|^2}{2\lambda} - \frac{1}{\lambda} \left(\frac{\|\cdot\|^2}{2} + \lambda f \right)^*(x), \quad (2)$$

$$f^*(s) = \frac{\|s\|^2}{2} - \lambda M_\lambda \left(\frac{1}{\lambda} f - \frac{\|\cdot\|^2}{2\lambda} \right)(s), \quad (3)$$

where $f : \mathbb{R}^d \rightarrow \mathbb{R} \cup \{+\infty\}$, and $\lambda > 0$. Hence, the PLQ algorithms can be extended to nonconvex functions as soon as one can compute the convex envelope of a PLQ function.

Computing the convex envelope of a PLQ univariate function f from Eq. 1 is a nonlinear nonconvex nonsmooth optimization problem in $\mathbb{R}^2$. (It is in fact a quadratically constrained quadratic programming problem.) In our very specific setting, though, the function f is a nonsmooth piecewise quadratic univariate function which is continuous on the interior of its domain, and our two specialized algorithms not only guarantee the global optimality of the solution but also outperform general optimization algorithms.

From now on we consider a PLQ function $f : \mathbb{R} \rightarrow]-\infty, +\infty]$ defined by $f(x) = a_i x^2 + b_i x + c_i$ when $x_i < x \leq x_{i+1}$ for $i = 0, \dots, n$ with $-\infty = x_0 < x_1 < \dots < x_n < x_{n+1} = +\infty$, $a_i, b_i \in \mathbb{R}$, and $c_i \in]-\infty, +\infty]$. (Note that c_i can take the value $+\infty$ to allow us to model function with bounded domains.) Since f is a PLQ function it is assumed to be continuous on the interior of its domain. We will use the notation f_i to denote the restriction of f on $[x_i, x_{i+1}]$ i.e. $f_i(x) = a_i x^2 + b_i x + c_i$ when $x \in [x_i, x_{i+1}]$ and $f_i(x) = +\infty$ everywhere else. Then $f = \min_i f_i$.

3 Computing the Convex Envelope Through Convex Analysis

We present here an algorithm, that we named `plq_coSplit`, to compute the convex envelope of an univariate PLQ function. The idea is to use the formula for the conjugate of the maximum

$$[\max(f_1, f_2)]^* = \text{co min}(f_1^*, f_2^*)$$

to obtain

$$\text{co min}(\text{co } f_1, \text{co } f_2) = [\max(f_1^*, f_2^*)]^* \quad (4)$$

where f_1, f_2 are lower semicontinuous (lsc) proper functions. Hence, if we decompose a PLQ function f on $\mathbb{R}$ as $f = \min f_i$ where $f_i(x) = f(x)$ on $[x_i, x_{i+1}]$ and $+\infty$ everywhere else, we obtain

$$\text{co } f = \text{co min}_i f_i = \text{co min}_i \text{co } f_i = [\max_i f_i^*]^*.$$

The only equality that needs to be proven is $\text{co min } f_i = \text{co min co } f_i$, which holds in a more general framework by [12, Proposition X.2.6.1]. Note that we reduce the computation of the convex envelope on $\mathbb{R}$ to computing the envelope $\text{co } f_i$ of the restriction f_i of a (nonconvex) quadratic function to an interval (a trivial task), computing the conjugate of a (simple convex PLQ function) $\text{co } f_i$, computing the maximum of n PLQ functions, and then another conjugate computation (of a convex function). We thereby obtain Algorithm 1 detailed below.

Algorithm 1: plq_coSplit

Input: f a PLQ function

Output: $\text{co } f$ a PLQ function

```

1 begin
2   Split  $f$  into  $f_i$ ;
3   Compute  $\text{co } f_i$  for each  $i = 0, \dots, n$  by replacing any  $f_i$  with  $a_i < 0$  with the line going
      through  $(x_i, f_i(x_i))$  and  $(x_{i+1}, f_i(x_{i+1}))$ ; any  $f_i$  with  $a_i \geq 0$  is convex so  $\text{co } f_i = f_i$ ;
4   Compute  $f_i^* = (\text{co } f_i)^*$  using the PLQ conjugate algorithm for convex functions;
5   Compute  $\max f_i^*$  directly (it is a convex function);
6   Compute the conjugate of  $\max f_i^*$  to obtain  $\text{co } f$ ;
7 end
```

While Algorithm 1 looks simpler than Algorithm 2 below, it relies on higher level algorithms, like computing the maximum and the conjugate. Since computing the conjugate can be done in linear time, lines 2–4, and 6 run in linear time. However, line 5 requires quadratic time to run (each f_i^* has at most three pieces but we need to compute the maximum of n of them). As for the space, we always store a linear number of pieces.

Proposition 3.1 *The plq_coSplit algorithm runs in quadratic time and uses linear space.*

The advantage of Algorithm 1 is it requires minimum amount of effort to implement as soon as the conjugate, and max operators are implemented. The price of such minimal work is the higher computational cost. The next section proposes a linear time algorithm.

4 Computing the Convex Envelope Directly

First, we make the following observation on how f can fail to be convex. We note $f'_i(x_i)$ (resp. $f'_i(x_{i+1})$) the right (resp. left) derivative of f_i i.e. $f'_i(x_i) = 2a_i x_i + b_i$ (resp. $f'_i(x_{i+1}) = 2a_i x_{i+1} + b_i$).

Fact 4.1 *Assume f is a PLQ function defined as above. Then f is nonconvex if, and only if, there is $i \in \{0, \dots, n\}$ with $a_i < 0$ or there is $j \in \{1, \dots, n\}$ with $f'_{j-1}(x_j) > f'_j(x_j)$.*

Next we note that the values of f on the intervals $]x_0, x_1]$ and $[x_n, x_{n+1}[$ determine whether f is proper or not.

Proposition 4.2 *Assume f is a PLQ function as defined above. Then $\text{co } f$ is identically $-\infty$ if, and only if, one or more of the following conditions are satisfied*

- (i) c_0 is finite on $]x_0, x_1]$ with $a_0 < 0$,
- (ii) c_n is finite on $[x_n, x_{n+1}[$ with $a_n < 0$,
- (iii) f is finite on $]x_0, x_1] \cup [x_n, x_{n+1}[$ with $a_0 = a_n = 0$ and $f'_0(x_1) > f'_n(x_n)$.

In all the other cases, $\text{co } f$ is a proper PLQ function.

Based on the previous results, we propose Algorithm 2, that we name `plq_coDirect`, to compute $\text{co } f$. The algorithm relies on the computation of functions which are the minimum of convex functions. When the function is piecewise linear with bounded domain, the algorithm reduces to the one dimensional Beneath-Beyond convex hull algorithm [8]. However, when the domain is unbounded or the function is piecewise quadratic with no restriction on the domain, the algorithm considers different cases. The core difference lies in computing the convex envelope of the restriction of two quadratic functions (Q-Q case), a quadratic and a linear (Q-L and L-Q case) instead of the single case of the restriction of two linear functions (L-L case). Moreover, the algorithm detects the quadratic-linear-quadratic (Q-L-Q) case directly and reduces it to a Q-Q case when the linear part does not appear in the convex envelope. While directly handling the Q-L-Q case destroys the simplicity of the Beneath-Beyond algorithm, it is necessary to ensure the linear-time complexity of the algorithm.

Algorithm `plq_coDirect` starts with handling improper convex envelopes on lines 2–4. Then concave parts are replaced by their convex envelopes on lines 5–9. The main loop, on lines 10–24, goes through each interval in the partition from left to right. The convex envelope of the leftmost part with the part just before is computed on line 12. Based on the result of that computation, the algorithm either considers the next part on line 21 or recursively considers the previous part on line 18. Note the newly computed parts need to be re-indexed on lines 13 (resp. line 17) when a new partition point is added (resp. when we need to consider previous parts). Figure 1 illustrates the algorithm. Figure 1c illustrates line 18 and shows that a single pass may not produce a convex function.

Algorithm 2: plq_coDirect**Input:** f a PLQ function**Output:** $\text{co } f$ a PLQ function

```

1 begin
2   if  $a_0 < 0$  or  $a_n < 0$  or  $(a_0 = a_n = 0 \text{ and } f'_0(x_1) > f'_n(x_n))$  then
3     | return  $\text{co } f \equiv -\infty$ 
4   end
5   for  $i=0$  to  $n$  do
6     | // Replace concave parts with their convex envelope
7     | if  $a_i < 0$  then
8       | | replace  $f$  on  $[x_i, x_{i+1}]$  with the linear function going through  $(x_i, f(x_i))$  and
9       | |  $(x_{i+1}, f(x_{i+1}))$ 
10    | end
11  end
12  for  $k=2$  to  $n$  do //main loop
13    | if Q-L-Q configuration is detected, reduce it to Q-Q configuration
14    | Compute  $\text{co}(f_{k-1}, f_k)$  the convex envelope of  $f$  restricted to  $[x_{k-1}, x_{k+1}]$ ;
15    | if a new partition point is created then
16    | | // convex envelope found on  $]x_0, x_{k+1}]$ 
17    | | re-index  $k$ ;
18    | |  $k = k + 1$ ;
19    | else
20    | | if  $f'_{k-1}(x_k) > f'_k(x_k)$  then
21    | | | re-index  $k$ ;
22    | | | // backtrack
23    | | | do not increase  $k$  i.e. the next iteration of the loop will consider  $[x_{k-1}, x_{k+1}]$ 
24    | | | with split point at  $x_{k-1}$ 
25    | | else
26    | | | // convex envelope found on  $]x_0, x_{k+1}]$ 
27    | | |  $k = k + 1$ ;
28    | | end
29    | end
30  end
31 end

```

The only remaining step required by the algorithm is the ability to compute the convex envelope $\text{co}(f_k, f_{k+1})$ of a continuous piecewise quadratic function defined on two connected intervals. The algorithm considers four cases: Q-Q, Q-L, L-Q, and L-L.

First, we consider the Q-Q case. Given two strictly convex quadratic functions f_{i-1} and f_i with domain $[x_{i-1}, x_{i+1}]$, partitioned at x_i , the following system of equations with unknowns x_α and x_β , has an explicit solution since it reduces to finding the roots of a quadratic polynomial.

$$\begin{aligned}
 f'_{i-1}(x_\alpha) &= f'_i(x_\beta) \\
 -f'_{i-1}(x_\alpha)x_\alpha + f_{i-1}(x_\alpha) &= -f'_i(x_\beta)x_\beta + f_i(x_\beta) \\
 x_{i-1} &\leq x_\alpha \leq x_i \\
 x_i &\leq x_\beta \leq x_{i+1}.
 \end{aligned}$$

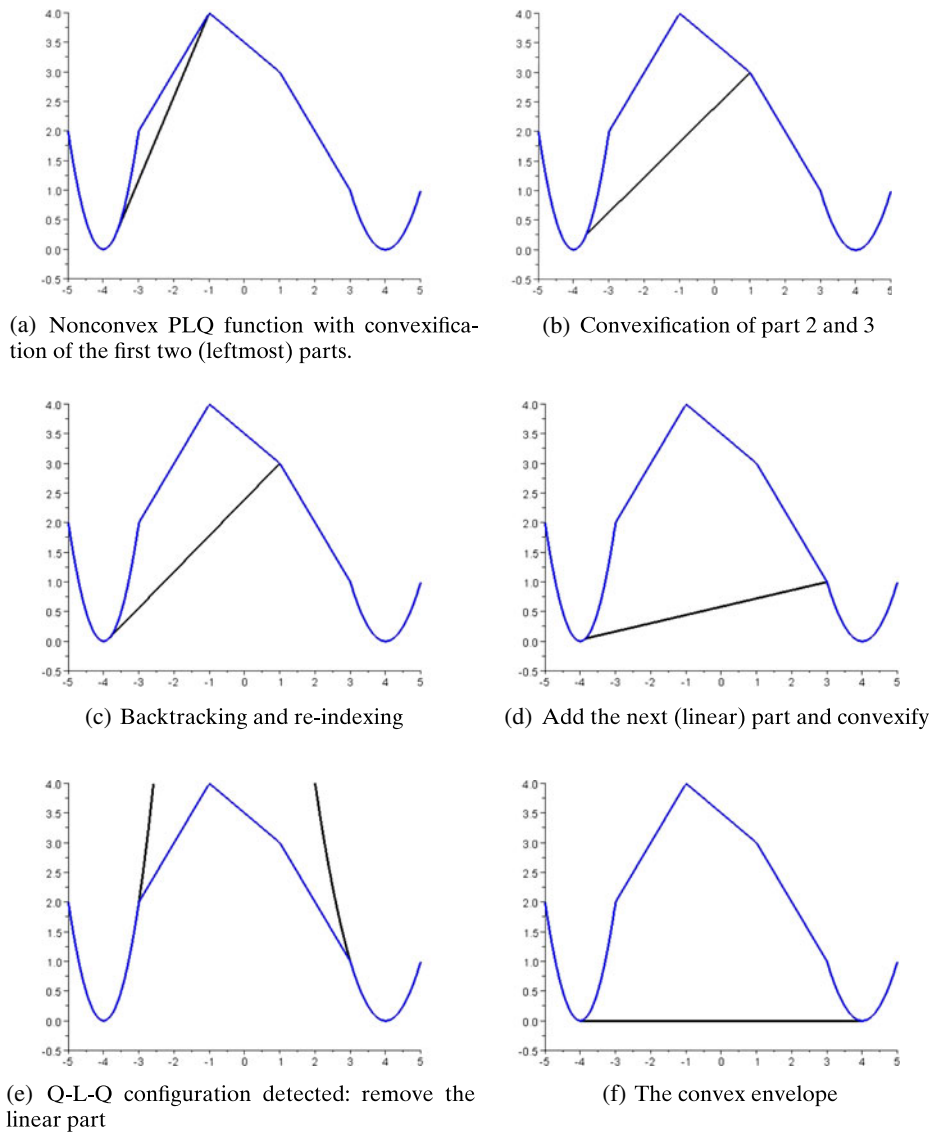


Fig. 1 The above sequence of images illustrates the steps of the PLQ convex hull algorithm. We see from Fig. 1c that it is necessary to backtrack at most once per iteration

The convex envelope $\text{co}(f_{i-1}, f_i)$ is given by

$$\text{co}(f_{i-1}, f_i)(x) = \begin{cases} f_{i-1}(x) & \text{if } x_{i-1} \leq x < x_\alpha, \\ f'_{i-1}(x_\alpha)(x - x_\beta) + f_i(x_\beta) & \text{if } x_\alpha \leq x \leq x_\beta, \\ f_i(x) & \text{if } x_\beta < x \leq x_{i+1}. \end{cases}$$

Figure 2 illustrates the Q-Q case. Note that the existence (and unicity) of the solutions is guaranteed by the strict convexity of the functions and the fact $f_{i-1}(x_i) = f_i(x_i)$.

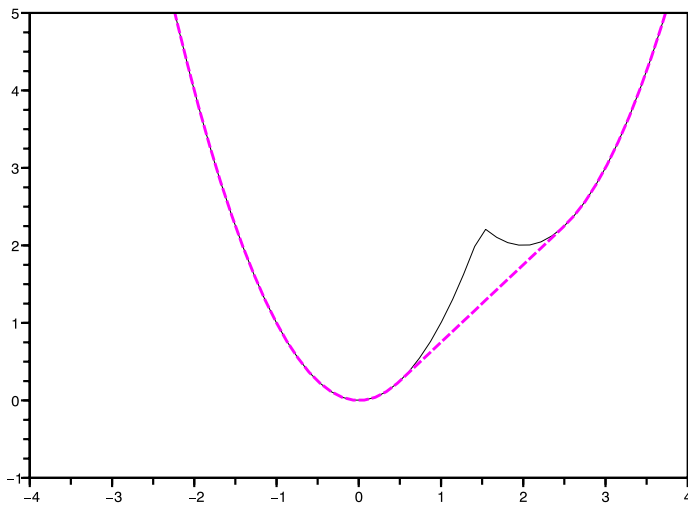


Fig. 2 The convex envelope of a piecewise “quadratic-quadratic” function. The “dashed” function is $\text{co } f(x)$, and the partition points x_α and x_β are where the affine function $f'_{i-1}(x_\alpha)(x - x_\beta) + f_i(x_\beta)$ bridges the gap between the left quadratic $f_{i-1}(x)$, and the right quadratic $f_i(x)$

Note also that the Q-Q case may introduce new points in the x_i partition. For example, in Fig. 2 we started with three points (x_{i-1} , x_i , and x_{i+1}) and ended up with four points (x_{i-1} , x_α , x_β , and x_{i+1}). That is why we reindex the parts to always have parts numbered from 0 to the current part.

For the L-Q case, we consider one linear function f_{i-1} and one strictly convex quadratic function f_i defined on the interval $[x_{i-1}, x_{i+1}]$ with $f_{i-1}(x_i) = f_i(x_i)$. We find an x_β such that

$$f'_{i-1}(x_\beta)(x - x_{i-1}) + f_{i-1}(x_{i-1}) = f'_i(x_\beta)(x - x_\beta) + f_i(x_\beta), \quad \forall x \in \mathbb{R},$$

$$x_i \leq x_\beta \leq x_{i+1}.$$

Then the convex envelope is given by

$$\text{co}(f_{i-1}, f_i)(x) = \begin{cases} f'_i(x_\beta)(x - x_{i-1}) + f_{i-1}(x_{i-1}) & \text{if } x_{i-1} \leq x \leq x_\beta, \\ f_i(x) & \text{if } x_\beta < x \leq x_{i+1}. \end{cases}$$

Figure 3 illustrates the L-Q case. Note that x_β may be $-\infty$ as shown on Fig. 3b.

The Q-L case is symmetric to the above and is handled similarly. The L-L case is handled as in the Beneath-Beyond algorithm for bounded intervals and is easily extended to unbounded intervals as illustrated by Fig. 4.

Finally, we handle the Q-L-Q case. It involves a linear function sandwiched between two quadratics when the linear part does not appear in the convex envelope (if the linear part appears in the convex envelope, it is a Q-L case followed by a L-Q case). In that configuration, a direct application of the algorithm triggers a convergence process that breaks the linear running time of the algorithm. Consider Fig. 1d. The function is composed of a quadratic, a linear, and a quadratic part. At that point, we compute the convex envelope of the linear-quadratic part, and obtain a quadratic-linear-quadratic function. However, the first breakpoint is not a minimum

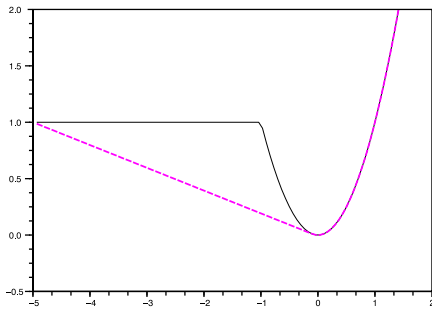
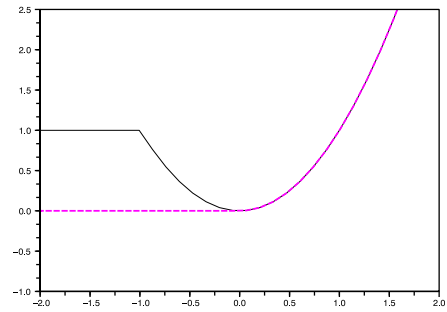
(a) Case x_β is finite.(b) Case $x_\beta = -\infty$.

Fig. 3 The convex envelope of a piecewise “linear-quadratic” function. The “dashed” function is $\text{co } f(x)$, and the partition point at x_β is where the affine function $f'_{i-1}(x_\beta)(x - x_{i+1}) + f_i(x_{i+1})$ on the left joins the quadratic $f_i(x)$ on the right

so the quadratic-linear part is not convex, we need to backtrack. After we convexify the first two parts, we once again have a quadratic-linear-quadratic function. Since none of the breakpoints are at a minimum, the function is not convex. The algorithm then iterates and converges toward the convex envelope. However, its linear running

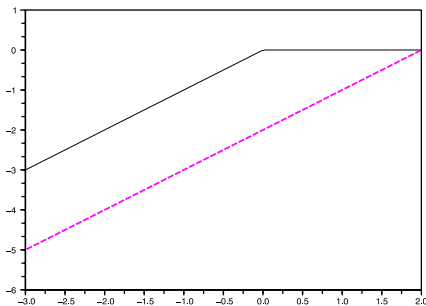
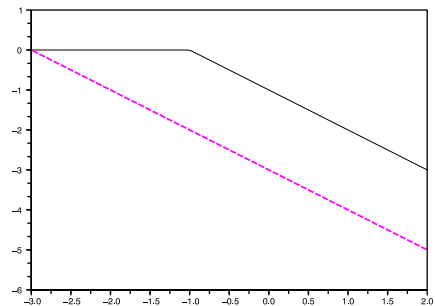
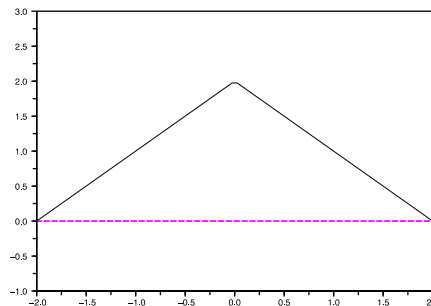
(a) f is bounded from the right.(b) f is bounded from the left.(c) f is bounded from both sides with $\text{dom } f = [-2, 2]$.

Fig. 4 The convex envelope of a piecewise “linear-linear” function

time is not guaranteed. In practice the convergence is too fast to plot. Consider the PLQ function

$$f(x) = \begin{cases} (x+1)^2 & \text{if } x < -0.5, \\ 0.25 & \text{if } x < 0.5, \\ (x-1)^2 & \text{if } x \geq -0.5, \end{cases}$$

plotted on Fig. 5. The algorithm generates the following sequence of PLQ matrices

$$\begin{bmatrix} -0.9142136 & 1. & 2. & 1. \\ 0.5 & 0. & 0.1715729 & 0.1642136 \\ \infty & 1. & -2. & 1. \end{bmatrix},$$

$$\begin{bmatrix} -0.9142136 & 1. & 2. & 1. \\ 0.9980768 & 0. & -0.0038465 & 0.0038428 \\ \infty & 1. & -2. & 1. \end{bmatrix},$$

$$\begin{bmatrix} -0.9999991 & 1. & 2. & 1. \\ 0.9980768 & 0. & 0.0000019 & 0.0000019 \\ \infty & 1. & -2. & 1. \end{bmatrix},$$

$$\begin{bmatrix} -0.9999991 & 1. & 2. & 1. \\ 1. & 0. & -4.284E-13 & 4.284E-13 \\ \infty & 1. & -2. & 1. \end{bmatrix},$$

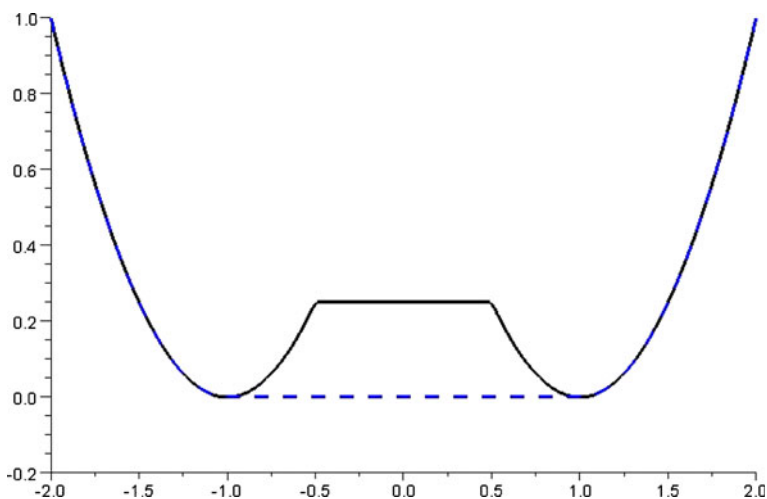


Fig. 5 Q-L-Q case: the convex envelope of a piecewise “quadratic-linear-quadratic” function can be reduced to the Q-Q case by removing the linear part in between. The “dashed” function is $\text{co } f(x)$

to obtain the convex envelope

$$\begin{bmatrix} -1. & 1. & 2. & 1. \\ 1. & 0. & 0. & 0. \\ \infty & 1. & -2. & 1. \end{bmatrix}.$$

The solution is to explicitly detect the Q-L-Q case, remove the linear piece and compute its convex envelope with the Q-Q case. The iterates for the above example become.

$$\begin{bmatrix} -0.9142136 & 1. & 2. & 1. \\ 0.5 & 0. & 0.1715729 & 0.1642136 \\ \infty & 1. & -2. & 1. \end{bmatrix},$$

$$\begin{bmatrix} 0. & 1. & 2. & 1. \\ \infty & 1. & -2. & 1. \end{bmatrix},$$

and finally

$$\begin{bmatrix} -1. & 1. & 2. & 1. \\ 1. & 0. & 0. & 0. \\ \infty & 1. & -2. & 1. \end{bmatrix}.$$

Each of the cases, Q-Q, Q-L, L-Q, L-L, Q-L-Q, is handled in constant time. Hence, the resulting algorithm runs in linear time.

We note that the `plq_coDirect` algorithm has optimal space complexity using a list structure to store piecewise functions. It also runs in optimal time complexity.

Proposition 4.3 *The `plq_coDirect` algorithm runs in linear time and space.*

Proof When the function f is piecewise linear with bounded domain, this is the Beneath-Beyond algorithm, which is known to run in linear time [8]. Unbounded domains are handled by Proposition 4.2. So the only remaining case is when f contains a piece with $a_i > 0$ i.e. is a strictly convex quadratic function on an interval $[x_i, x_{i+1}]$.

We count the number of times a part $[x_i, x_{i+1}]$ is considered. If the function f restricted to that part is linear, then the part is added once and maybe removed once. So it is considered at most twice. If the restriction is concave ($a_i < 0$), it is replaced by a linear part and is handled as in the previous case. The only remaining case is when the restriction is strictly convex ($a_i > 0$). Then it is added once, removed at most once, but can also be split with the addition of a new partition point. However, there can only be a linear number of new partition points as at most two such points are created at each iteration of the algorithm. Consequently, the algorithm runs in linear time. $\square$

5 Numerical Experiments

All numerical experiments were performed on an IBM T60 with two cores running at 1.8 GHz and 2 GB memory under Scilab 5.1.0.

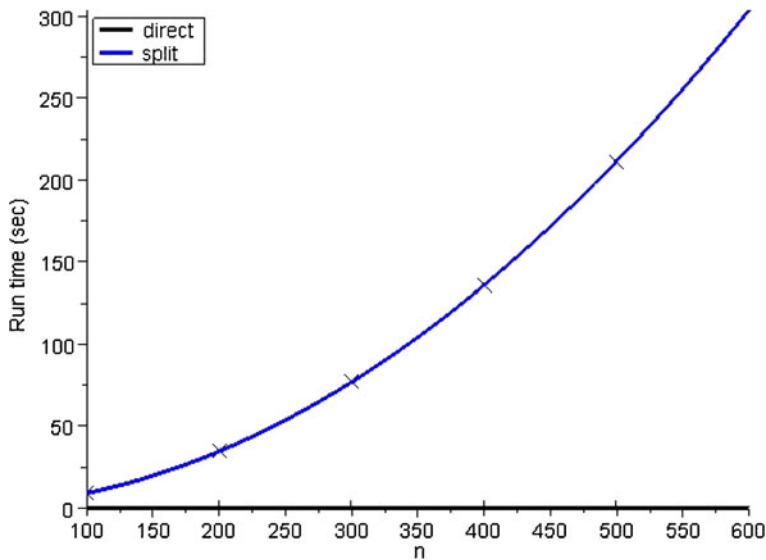
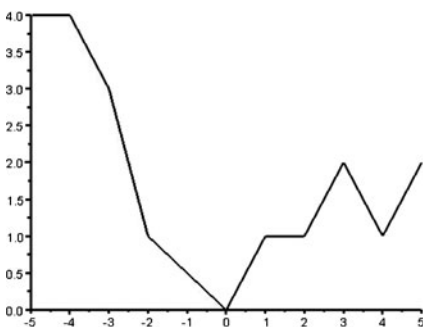


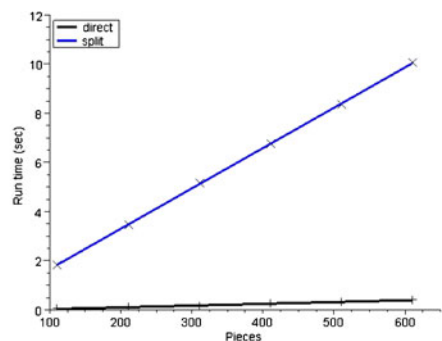
Fig. 6 Running times for the plq_coDirect and the plq_coSplit algorithms for the function $f(x) = x^2/2$ on interval $[-n/2, n/2]$ using n pieces

Numerical experiments that validate the linear complexity of both algorithms plq_coDirect and plq_coSplit are presented in Figs. 6 and 7b. Figure 6 illustrates the quadratic worst case complexity for plq_coSplit when $f(x) = x^2/2$ on $[-n/2, n/2]$.

On the other hand, Fig. 7b shows plq_coSplit runs in linear time when applied to the PLQ function shown in Fig. 7a. In both cases, the plq_coDirect algorithm clearly outperforms the plq_coSplit algorithm. This fact is not surprising since plq_coSplit uses several high level algorithms that could be further tuned (especially our computation of the maximum). Both algorithms have their own advantages: plq_coDirect is faster in practice but requires a tedious implementation to handle



(a) A PLQ function with many pieces.



(b) Runtime of plq_coDirect and plq_coSplit.

Fig. 7 Runtime performance of plq_coDirect and plq_coSplit algorithms for a piecewise linear PLQ function

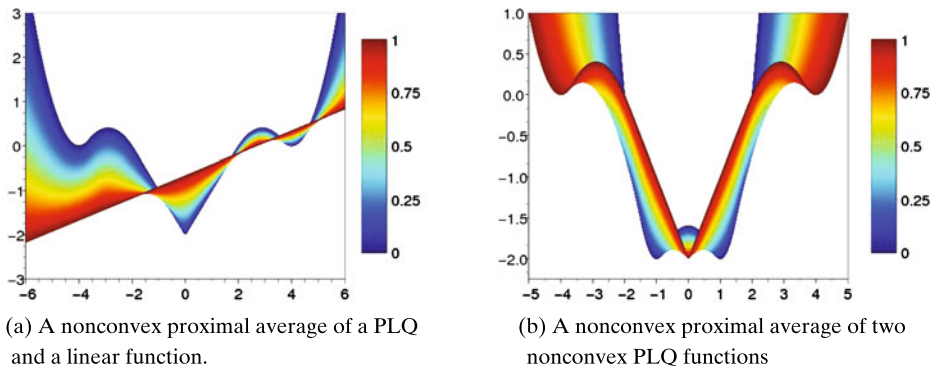


Fig. 8 Nonconvex proximal averages computed using the `plq_coDirect` algorithm coupled with (convex) PLQ algorithms. Each curve with parameter λ is displayed with a different color

several special cases, while `plq_coSplit` is easier to implement (since it builds on previous algorithms) at the price of a slower (quadratic) running time.

We conclude this section with a couple of pictures of nonconvex proximal averages. The proximal average [1, 2] of two functions f_0 and f_1 with parameter λ is defined by

$$\mathcal{P}(f_0, f_1; \lambda) := \left[(1 - \lambda) \left(f_0 + \frac{1}{2} |\cdot|^2 \right)^* + \lambda \left(f_1 + \frac{1}{2} |\cdot|^2 \right)^* \right]^* - \frac{1}{2} |\cdot|^2.$$

It has been extended to nonconvex functions with a slightly different definition [11]. Figure 8a and b show nonconvex proximal averages computed using the `plq_coDirect` algorithm.

6 Conclusion

We extended the computational convex analysis (CCA) numerical toolbox to nonconvex functions by adding two algorithms to compute the convex envelope. The first one is easy to implement but has a quadratic complexity, while the second has optimal linear time complexity but requires more effort to implement. Both algorithms outperform general nonsmooth nonconvex nonlinear optimization algorithms on two key points: they guarantee the global optimality of the solution, and they run much faster.

We note the following directions for future research. New convex analysis transforms, like the kernel average [3], have been recently introduced that require extension of current algorithms. While some transforms like the Moreau envelope and the Fenchel conjugate can be computed efficiently for convex and nonconvex functions, the efficient computation for others is limited to convex functions. The inf-convolution is one such operator, although sub-quadratic algorithms exist for its computation their optimality is unknown [4].

There are also important applications that require the computation of the closest closed convex function of a nonconvex function i.e. to compute the projection on the

cone of closed convex functions [15]. As far as we know, such projections cannot be achieved using the convex envelope.

Finally, the search for a convex hull algorithm for bivariate PLQ functions is the subject of active research. Algorithm `plq_coSplit` will be difficult to extend as the maximum of two bivariate PLQ functions is not always a PLQ function e.g. $\max(x^2, 1)$ is equal to x^2 outside the unit circle but to 1 inside. Since the unit circle is not polygonal, the maximum is not a PLQ function. Another challenge is that even adding two bivariate PLQ functions can no longer be achieved in linear time as it is equivalent to the map overlay problem which has quadratic worst-case time complexity [7].

Acknowledgements Yves Lucet was partially supported by a Discovery grant from the Natural Sciences and Engineering Research Council of Canada. The authors would like to thank M. Trienis for a preliminary implementation of algorithm `plq_coDirect` [27].

References

1. Bauschke, H.H., Goebel, R., Lucet, Y., Wang, X.: The proximal average: basic theory. *SIAM J. Optim.* **19**, 768–785 (2008)
2. Bauschke, H.H., Lucet, Y., Trienis, M.: How to transform one convex function continuously into another. *SIAM Rev.* **50**, 115–132 (2008)
3. Bauschke, H.H., Wang, X.: The kernel average of two convex functions and its application to the extension and representation of monotone operators. *Trans. Am. Math. Soc.* **361**, 5947–5965 (2009)
4. Bremner, D., Chan, T.M., Demaine, E.D., Erickson, J., Hurtado, F., Iacono, J., Langerman, S., Taslakian, P.: Necklaces, convolutions, and $X + Y$. In: *Algorithms—ESA 2006. Lecture Notes in Computer Science*, vol. 4168, pp. 160–171. Springer, Berlin (2006)
5. Brenier, Y.: Un algorithme rapide pour le calcul de transformées de Legendre–Fenchel discrètes. *C. R. Acad. Sci. Paris Sér. I Math.* **308**, 587–589 (1989)
6. Corrias, L.: Fast Legendre–Fenchel transform and applications to Hamilton–Jacobi equations and conservation laws. *SIAM J. Numer. Anal.* **33**, 1534–1558 (1996)
7. de Berg, M., van Kreveld, M., Overmars, M., Schwarzkopf, O.: *Algorithms and applications*. In: *Computational Geometry*, 3rd edn. Springer, Berlin (2008)
8. Edelsbrunner, H.: *Algorithms in Combinatorial Geometry*. EATC Monographs on Theoretical Computer Science. Springer (1987)
9. Felzenszwalb, P.F., Huttenlocher, D.P.: *Distance Transforms of Sampled Functions*. Tech. Rep. TR2004-1963, Cornell Computing and Information Science (2004)
10. Gardiner, B., Lucet, Y.: *Graph-Matrix Calculus for Computational Convex Analysis. Fixed-Point Algorithms for Inverse Problems in Science and Engineering*. Springer Verlag series Optimization and Its Application (2010, in press)
11. Hare, W.: A proximal average for nonconvex functions: a proximal stability perspective. *SIAM J. Optim.* **20**, 650–666 (2009)
12. Hiriart-Urruty, J.-B., Lemaréchal, C.: *Convex analysis and minimization algorithms*. In: *Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences]*, vols. 305–306. Springer, Berlin (1993); Vol I: Fundamentals. Vol II: Advanced Theory and Bundle Methods
13. Hiriart-Urruty, J.-B., Lucet, Y.: Parametric computation of the Legendre–Fenchel conjugate. *J. Convex Anal.* **14**, 657–666 (2007)
14. Koch, V., Johnstone, J., Lucet, Y.: Convexity of the proximal average. *J. Optim. Theory Appl.* (2010, in press)
15. Lachand-Robert, T., Oudet, É.: Minimizing within convex bodies using a convex hull method. *SIAM J. Optim.* **16**, 368–379 (2005, electronic)
16. Laraki, R., Lasserre, J.B.: Computing uniform convex approximations for convex envelopes and convex hulls. *J. Convex Anal.* **15**, 635–654 (2008)
17. Lucet, Y.: A fast computational algorithm for the Legendre–Fenchel transform. *Comput. Optim. Appl.* **6**, 27–57 (1996)

18. Lucet, Y.: Faster than the fast Legendre transform. The linear-time Legendre transform. *Numer. Algorithms* **16**, 171–185 (1997)
19. Lucet, Y.: A linear Euclidean distance transform algorithm based on the linear-time Legendre transform. In: *Proceedings of the Second Canadian Conference on Computer and Robot Vision (CRV 2005)*, pp. 262–267, Victoria BC. IEEE Computer Society Press (2005)
20. Lucet, Y.: Fast Moreau envelope computation I: numerical algorithms. *Numer. Algorithms* **43**, 235–249 (2006)
21. Lucet, Y.: What shape is your conjugate? A survey of computational convex analysis and its applications. *SIAM J. Optim.* **20**, 216–250 (2009)
22. Lucet, Y., Bauschke, H.H., Trienis, M.: The piecewise linear-quadratic model for computational convex analysis. *Comput. Optim. Appl.* **43**, 95–118 (2009)
23. Moreau, J.-J.: Proximité et dualité dans un espace Hilbertien. *Bull. Soc. Math. Fr.* **93**, 273–299 (1965)
24. Noullez, A., Vergassola, M.: A fast Legendre transform algorithm and applications to the adhesion model. *J. Sci. Comput.* **9**, 259–281 (1994)
25. Rockafellar, R.T., Wets, R.J.-B.: *Variational Analysis*. Springer, Berlin (1998)
26. She, Z.-S., Aurell, E., Frisch, U.: The inviscid Burgers equation with initial data of Brownian type. *Commun. Math. Phys.* **148**, 623–641 (1992)
27. Trienis, M.: *Computational Convex Analysis: From Continuous Deformation to Finite Convex Integration*. Master's Thesis, University of British Columbia (2007)